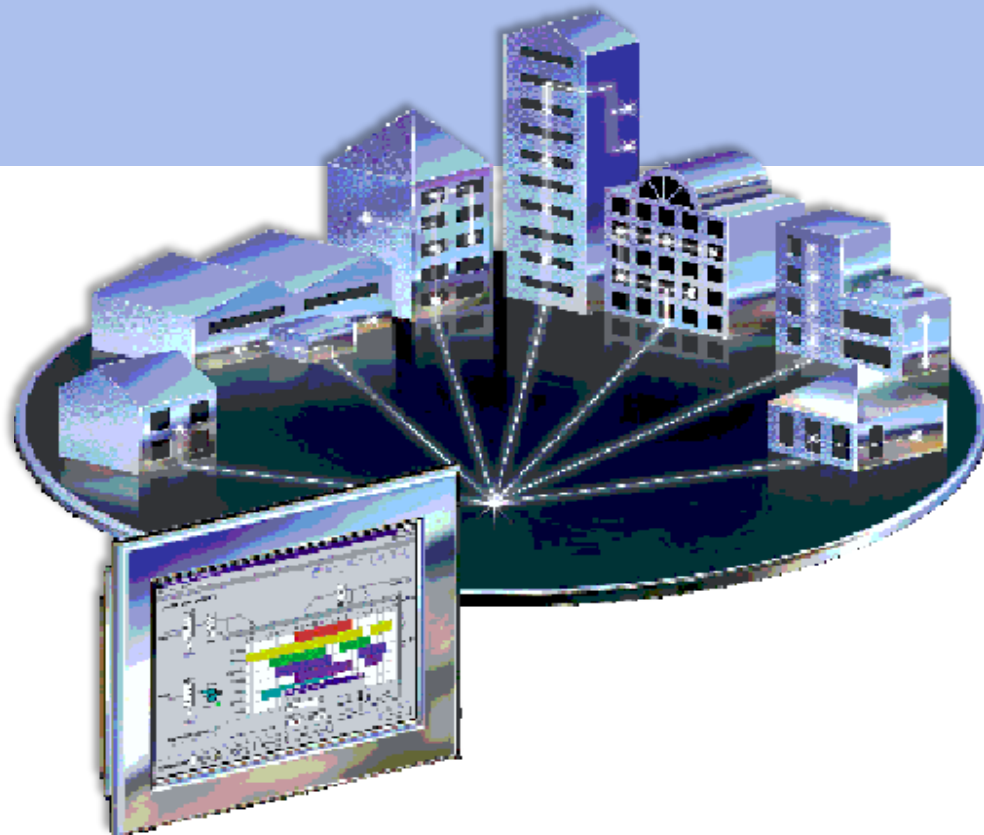


INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO



Práctica 6:

SERVICIOS WEB



SISTEMAS DISTRIBUIDOS

Profesor || Chadwick Carreto Arellano
Grupo || 7CM1
Alumno || Areli Alejandra Guevara Badillo

Abril 15, 2026

EL ROL DE LOS SERVICIOS WEB EN LA INGENIERÍA DE SOFTWARE

En la actualidad, el desarrollo de sistemas se fundamenta en la integración de aplicaciones distribuidas capaces de comunicarse de manera eficiente, segura y estandarizada. Los **servicios web** se han consolidado como un pilar esencial, permitiendo la interoperabilidad entre sistemas heterogéneos mediante el uso de protocolos abiertos y tecnologías universales como **HTTP, JSON y XML**.

Este paradigma ha transformado el diseño de software:

- **De Monolitos a Microservicios:** Las aplicaciones ya no se conciben como bloques cerrados, sino como un conjunto de servicios independientes con responsabilidades específicas.
- **Interfaces Definidas:** La funcionalidad se expone a través de interfaces claras, lo que potencia la reutilización de código.
- **Escalabilidad:** Este enfoque facilita el crecimiento horizontal y la integración fluida con sistemas internos y externos.

El Estilo Arquitectónico REST

Propuesto por Roy Fielding en el año 2000, **REST (Representational State Transfer)** define principios que promueven la simplicidad y la independencia entre cliente y servidor.

Principios Fundamentales de REST:

- **Identificación de Recursos:** Utiliza **URIs** (Uniform Resource Identifiers) para localizar c/entidad.
- **Interfaz Uniforme:** Manipulación de recursos mediante métodos estándar de HTTP.
- **Modelo Sin Estado (Stateless):** Cada petición es independiente y contiene toda la información necesaria para ser procesada, lo que optimiza la eficiencia del servidor.

Debido a su ligereza, REST es la base de la mayoría de las **APIs modernas**, permitiendo el intercambio de datos en aplicaciones móviles, plataformas web y entornos de microservicios sin dependencia de lenguajes o sistemas operativos específicos.

Estructuración del Software: La Arquitectura en Capas

Un servicio web robusto no depende solo de su protocolo, sino de su organización interna. La **arquitectura en capas** es la práctica estándar para aplicar el principio de **Separación de Responsabilidades (Separation of Concerns)**.

Capa	Función Principal
Presentación	Gestiona la interacción con el usuario o cliente final.
Aplicación	Coordina los flujos de la lógica de negocio.
Dominio	Define las entidades y las reglas fundamentales del sistema.
Infraestructura	Maneja aspectos técnicos como persistencia (BD) y conexiones de red.

Este desacoplamiento permite sustituir componentes (como el motor de base de datos o el framework web) sin alterar la lógica central del negocio, facilitando el mantenimiento y las pruebas unitarias.

Evolución hacia la Clean Architecture

En años recientes, propuestas como la **Clean Architecture** de Robert C. Martin ("Uncle Bob") han llevado la modularidad a un nivel superior. Su núcleo es la **Inversión de Dependencias**:

- Las reglas de negocio (núcleo) son totalmente independientes de los detalles tecnológicos.
- Los frameworks, bases de datos e interfaces son considerados "detalles externos" que pueden ser reemplazados sin afectar la estabilidad del sistema.

Herramientas de Ecosistema: ORMs y Contenedores

El desarrollo moderno se apoya en herramientas que garantizan la robustez y la portabilidad:

1. **ORMs (Object-Relational Mappers)**: Herramientas como *SQLAlchemy* o *Hibernate* mapean objetos de programación a tablas relacionales, eliminando la necesidad de SQL manual y simplificando la persistencia de datos.
2. **Contenedores (Docker)**: Han revolucionado el despliegue al ofrecer entornos aislados y reproducibles. Esto garantiza la paridad entre los entornos de desarrollo y producción, eliminando el clásico problema de "en mi máquina sí funciona".

PLANTEAMIENTO DEL PROBLEMA

En el aprendizaje y dominio de la arquitectura de servicios web, uno de los desafíos principales es comprender de manera práctica la interacción entre un cliente y un servidor mediante el protocolo HTTP. A menudo, los desarrolladores noveles luchan por asimilar el concepto de un entorno "sin estado" (stateless), donde cada petición debe contener toda la información necesaria para ser procesada.

Las prácticas tradicionales, como los sistemas de inventario o las clásicas listas de tareas (To-Do lists), suelen introducir lógica de base de datos o requerimientos de negocio complejos que desvían la atención de los conceptos arquitectónicos fundamentales: el enrutamiento adecuado, el uso semántico de los verbos HTTP, y la estructura estandarizada de las respuestas en formato JSON. Se requiere un escenario de práctica directo, que permita manipular un estado interno de forma intuitiva, con retroalimentación inmediata sobre cómo las peticiones alteran dicho estado.

PROPUESTA DE SOLUCIÓN

Para abordar esta necesidad, se diseñó e implementó un **"API-gotchi"**: una mascota virtual cuyo estado vital y acciones son controlados exclusivamente a través de una API web.

Esta solución abstrae la lógica de negocio en reglas aritméticas simples (niveles de hambre, energía y felicidad) para concentrar el esfuerzo técnico en:

1. **Diseño de Endpoints:** Implementar métodos HTTP adecuados (GET para consultas de solo lectura, POST para acciones que mutan el estado).
2. **Gestión de Estado:** Administrar el ciclo de vida de un recurso en la memoria del servidor.
3. **Manejo de Excepciones:** Retornar códigos de estado HTTP (Status Codes) y mensajes de error coherentes cuando se violan las reglas de negocio.

Se eligió el framework **FastAPI** con **Python** por su alto rendimiento, validación automática de datos y su capacidad nativa para autogenerar documentación interactiva (Swagger UI).

Stack Tecnológico: Proyecto API-gotchi

Categoría	Tecnología	Función en el Proyecto	¿Por qué elegirla?
Lenguaje Base	Python (3.10+)	Lógica central, manejo de diccionarios y reglas de negocio del API-gotchi.	Sintaxis limpia, excelente manejo de tipos (Type Hints) y una curva de aprendizaje amigable.
Framework Web	FastAPI	Enrutamiento HTTP (GET, POST), validación de datos y manejo de códigos de estado.	Es uno de los frameworks más rápidos actuales, moderno y diseñado específicamente para construir APIs RESTful.
Servidor ASGI	Uvicorn	Levantar la aplicación y escuchar las peticiones HTTP entrantes.	Es el estándar de la industria para ejecutar aplicaciones asíncronas en Python de forma ultrarrápida.
Pruebas y Documentación	Swagger UI (Integrado en FastAPI)	Interfaz gráfica interactiva disponible en /docs para probar los endpoints.	Elimina la necesidad de configurar clientes externos en etapas tempranas. Se autogenera leyendo tu código.
Entorno de Desarrollo (IDE)	Visual Studio Code	Escritura de código, gestión de terminales y entorno virtual.	Ligero, con excelentes extensiones para Python (Pylance) que te avisan de errores antes de ejecutar el código.

DESARROLLO E IMPLEMENTACIÓN

El proyecto se construyó modularmente. A continuación, se detallan los fragmentos de código más relevantes de la implementación en main.py.

Estructura de Datos y Estado Inicial

Se utilizó un diccionario en memoria para simular el registro de la base de datos.

```
from fastapi import FastAPI, HTTPException

app = FastAPI(title="API-gotchi", description="Simulador de mascota virtual RESTful")

# Estado persistente en memoria durante la ejecución del servidor
mascota = {
    "nombre": "Pixel",
    "hambre": 50,      # Rango: 0 (lleno) a 100 (inanición)
    "felicidad": 50,   # Rango: 0 (triste) a 100 (feliz)
    "energia": 100,    # Rango: 0 (exhausto) a 100 (descansado)
    "estado": "vivo"   # Valores: "vivo", "fallecido"
}
```

Endpoint de Lectura (Idempotente)

Se implementó el verbo GET para consultar el recurso. Esta operación es segura y no modifica el estado del servidor.

```
@app.get("/estado")
def ver_mascota():
    """Devuelve el estado actual de tu API-gotchi."""
    return {"data": mascota, "status": "success"}
```

Endpoint de Mutación y Lógica de Negocio

Para las acciones, se utilizó el verbo POST. El siguiente fragmento muestra el endpoint /jugar, el cual incluye validaciones de estado previas a la mutación de los datos.

```
@app.post("/jugar")
def jugar():
    """Lógica para la acción de jugar."""
    # Validación 1: Estado vital
    if mascota["estado"] != "vivo":
        raise HTTPException(status_code=400, detail="Acción denegada: La mascota ha fallecido.")

    # Validación 2: Reglas de negocio (energía mínima)
    if mascota["energia"] < 20:
        raise HTTPException(
            status_code=422,
            detail="Pixel está muy cansado para jugar. Necesita dormir primero."
        )

    # Mutación del estado
    mascota["felicidad"] = min(100, mascota["felicidad"] + 15)
    mascota["energia"] = max(0, mascota["energia"] - 25)
    mascota["hambre"] = min(100, mascota["hambre"] + 10)

    # Evaluación de condiciones críticas
    if mascota["hambre"] ≥ 100:
        mascota["estado"] = "fallecido"
        return {
            "mensaje": "Alerta crítica: Pixel jugó demasiado, su hambre llegó al límite y falleció.",
            "data": mascota
        }

    return {"mensaje": "¡Pixel se divirtió mucho!", "data": mascota}
```

RESULTADOS

Ejemplos de Ejecución y Pruebas

Para validar el correcto funcionamiento, se simularon peticiones HTTP. A continuación, se muestran las trazas de ejecución.

Ejecución 1: Consulta del estado inicial (Método GET)

GET / Ver Mascota

Devuelve el estado actual de tu API-gotchi.

Parameters Cancel

No parameters

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
'http://127.0.0.1:8000/' \
-H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/
```

Server response

Code	Details
200	Response body <pre>{ "nombre": "Pixel", "hambre": 50, "felicidad": 50, "energia": 100, "estado": "vivo" }</pre> Download Response headers <pre>content-length: 75 content-type: application/json date: Wed, 15 Apr 2026 19:58:49 GMT server: uvicorn</pre>

Responses

Code	Description	Links
200	Successful Response	No links

Ejecución 2: Interacción exitosa (Método POST)

POST /alimentar Alimentar

Alimentar a la mascota reduce su hambre, pero da un poco de sueño.

Parameters

Cancel

No parameters

Execute

Clear

Responses

Curl

curl -X 'POST' \
 'http://127.0.0.1:8000/alimentar' \
 -H 'accept: application/json' \
 -d ''

Request URL

http://127.0.0.1:8000/alimentar

Server response

Code

Details

200

Response body

{
 "mensaje": "le diste de comer a Pixel. ¡Nam ñam!",
 "estado": {
 "nombre": "Pixel",
 "hambre": 30,
 "felicidad": 50,
 "energia": 85,
 "estado": "vivo"
 }
}

Response headers

content-length: 137
content-type: application/json
date: Wed, 15 Apr 2026 20:00:20 GMT
server: uvicorn

Responses

Code

Description

Links

200

Successful Response

No links

POST /dormir Dormir

Dormir recupera energía al máximo, pero da un poco de hambre.

Parameters

Cancel

No parameters

Execute

Clear

Responses

Curl

curl -X 'POST' \
 'http://127.0.0.1:8000/dormir' \
 -H 'accept: application/json' \
 -d ''

Request URL

http://127.0.0.1:8000/dormir

Server response

Code

Details

200

Response body

{
 "mensaje": "Pixel tomó una buena siesta. Zzz...",
 "estado": {
 "nombre": "Pixel",
 "hambre": 45,
 "felicidad": 50,
 "energia": 100,
 "estado": "vivo"
 }
}

Response headers

content-length: 135
content-type: application/json
date: Wed, 15 Apr 2026 20:00:53 GMT
server: uvicorn

Responses

Code

Description

Links

200

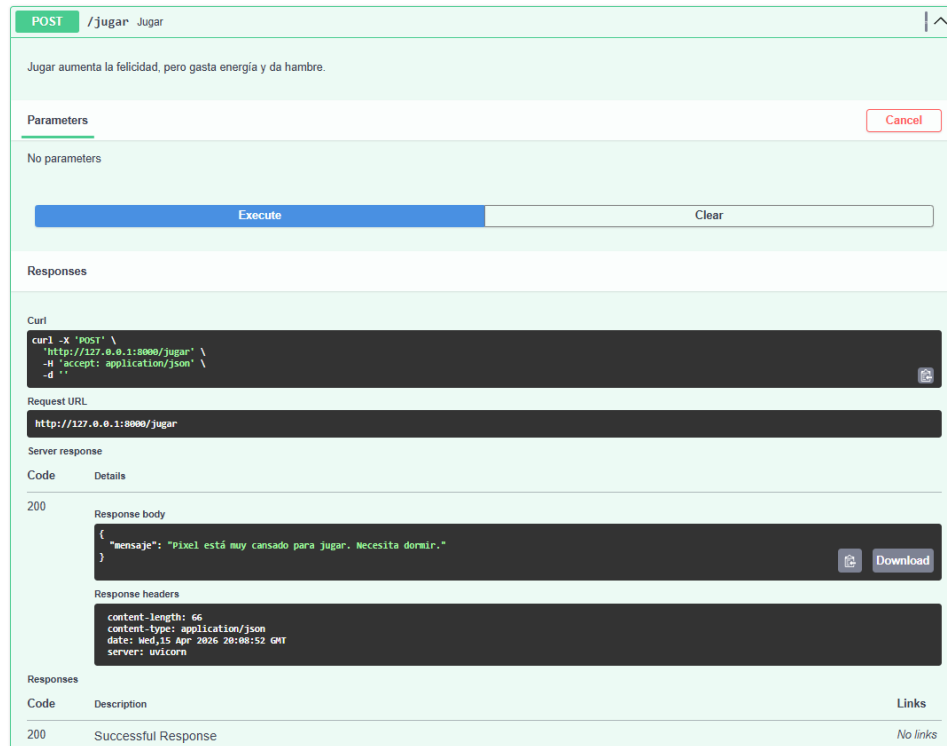
Successful Response

No links

7

Ejecución 3: Manejo de Errores - Regla de negocio no cumplida (Método POST)

Escenario: Se enviaron múltiples peticiones /jugar consecutivas hasta que la energía bajó a 0.



Resultados

El desarrollo concluyó con una API operativa y robusta. La separación de responsabilidades permitió validar que el cliente no dictamina el estado de la mascota, sino que solicita acciones, y es el servidor (la capa de aplicación) quien evalúa si dichas acciones son válidas según sus reglas de negocio. El uso de FastAPI generó un panel de documentación en /docs que permitió testear los casos de uso presentados en la sección anterior de forma visual e inmediata, simulando las peticiones sin necesidad de configurar clientes externos como Postman o Insomnia.

CONCLUSIONES

La implementación del API-gotchi demostró ser un modelo excelente para interiorizar los fundamentos del desarrollo backend. Las conclusiones técnicas más destacadas son:

- **Semántica HTTP:** Se comprobó empíricamente la diferencia entre métodos seguros (GET) y métodos de mutación (POST), y cómo estos interactúan con el estado del servidor.
- **Validación y Códigos de Estado:** El uso de excepciones (HTTPException) enfatizó la importancia de no responder siempre con un código 200 OK. Retornar códigos de la familia 4xx (como el 400 o 422) cuando el cliente solicita acciones inválidas es crucial para el diseño de una API predecible y profesional.

Como siguiente paso evolutivo, el diccionario en memoria debería ser reemplazado por una base de datos relacional (ej. SQLite) integrando un ORM (como SQLAlchemy), lo que garantizaría la persistencia de los datos entre reinicios del servidor.